



EdgeTimer: Adaptive Multi-Timescale Scheduling in Mobile Edge Computing with Deep Reinforcement Learning

Yijun Hao[†], Shusen Yang*, Fang Li[†], Yifan Zhang[†], Shibo Wang[†], and Xuebin Ren[†]
[†]*Xi'an Jiaotong University, China



汇报人: 1025040808王宇杰

场景

整个边缘调度过程可以划分为三层：

第一层：边缘-云调度，边缘云**服务部署**，各台边缘服务器中部署那些服务

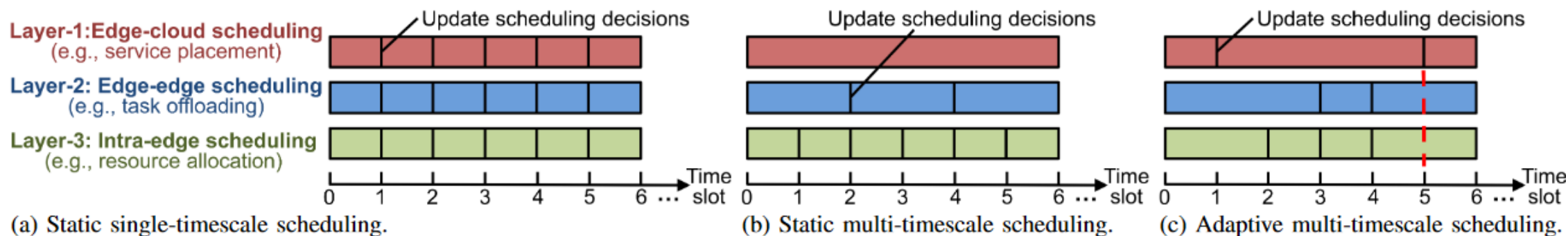
第二层：边缘-边缘调度，边缘**任务卸载**，到达的任务卸载到哪台服务器

第三层：边缘内调度，边缘内**资源分配**，为接受的任务分配多少资源处理

问题

大多数MEC调度器以**固定的时间间隔更新调度决策**，该时间间隔在所有三层中都是统一的，我们称之为**静态单时间尺度模式**。———较好的服务性能，**高昂的运营成本**。

较少的调度器在多个调度层上设置不同的更新间隔，但**每层的更新时间间隔仍然是固定的**，我们称之为**静态多时间尺度模式**。———无法适应环境变化。



如何自适应地确定多层调度决策的各自时间尺度，以实现运营成本和服务性能之间的权衡？

本文工作

提出了EdgeTimer，这是一种用于边缘云、边缘和边缘内调度的自适应多时间尺度调度框架。

它具有以下3个特征

对环境变化的**适应性**：每一层调度决策的**时间尺度都是时变的**，以适应在线任务请求的变化模式。

多层调度上的**异步性**：**时间尺度**的值是在不同的调度层上**独立**确定的。

边缘服务器的**自主性**：每个边缘服务器都需要在**本地确定**其时间尺度，而无需借助远程云的计算。

挑战和工作

- 1) 本文涉及三层决策，同时决策的**动作空间过大**，这使得学习任务对DRL而言变得复杂且难以处理。
- 2) 传统的DRL算法通常使用一个客户端进行**集中决策**。

To 1) 我们设计了一个**三层分层DRL框架**，将整个学习任务分解为三个独立的分层子任务，每个子任务都与一个单独的子目标相关联，并由DRL控制器处理。

To 2) 对于每一层DRL控制器，我们设计了一种**安全的多客户端DRL方法**来学习特定层的分布式策略。

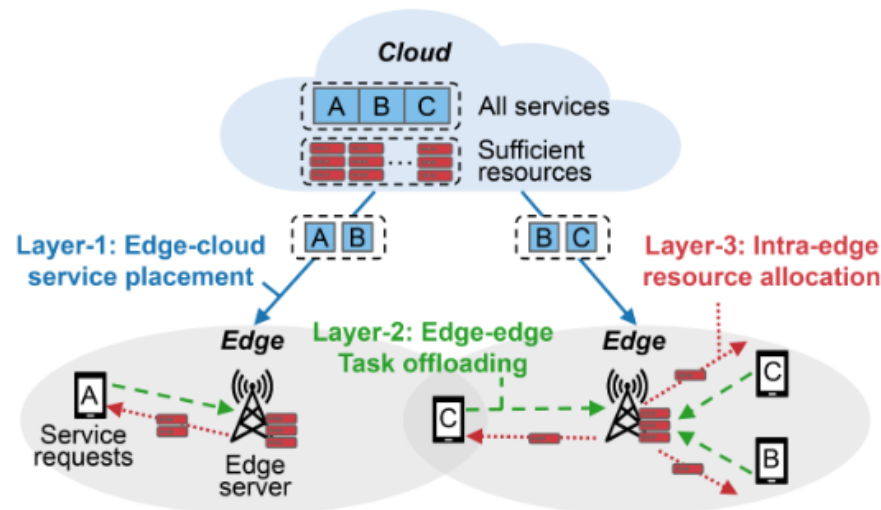
贡献

EdgeTimer是第一个自动控制多个调度决策的时间尺度以优化利润的工作。

- 1) 他们可以在不增加额外成本的前提下，通过提供高性能服务来直接提高利润。
- 2) EdgeTimer不会干扰服务提供商的当前调度方法。它只在必要时提供更新信号。服务提供商可以根据其丰富的经验决定是否依赖该信号。
- 3) 除了本文中的应用案例外，他们还可以通过简单地更改DRL控制器的状态和奖励设置，将EdgeTimer扩展到其他单层或多层调度决策。

系统模型和问题构建

MEC系统模型



N台边缘服务器 $\mathcal{N} = \{1, \dots, N\}$

离散时隙 $\mathcal{T} = \{0, 1, \dots, t, \dots\}$

云服务器 $\mathcal{O}$

- 1) 第一层：边缘-云服务部署：每个任务都有其需求的服务类型，所有的服务表示为 $\mathcal{S} = \{1, \dots, S\}$ 。用 $x_{i,s}(t)$ 表示在 t 时刻服务 s 是否部署在边缘服务器 i 上。
- 2) 第二层：边缘-边缘任务卸载：到达本地边缘服务器 i 的任务可以被卸载到缓存了相应服务的相邻边缘服务器 j 。
 $y_{i,j,s}(t)$ 表示需求服务 s 的任务是否在 t 时刻从边缘服务器 i 卸载到服务器 j 。
- 3) 第三层：边缘内资源分配：边缘服务器 i 在 t 时刻向需求服务 s 的任务分配了多少计算资源，符号表示为 $z_{i,s}(t)$ 。

系统模型和问题构建

利润模型（收入-成本）

服务提供商获得的利润可以通过从收入中减去运营成本来定义

收入

$$g_i(t) = \sum_{s \in \mathcal{S}} \mu_{i,s}(t) p_s(t), \quad (1)$$

$\mu_{i,s}(t)$ 是 t 时刻服务器 i 分配给 满足延迟预算的需求服务 s 的任务的计算资源总量

$p_s(t)$ 表示 t 时刻服务 s 的资源单价

运营成本

服务部署

放置服务 s 需要边缘服务器 i 从相邻边缘或远程云服务器 j 下载较大的数据, 这会产生运营成本 $C_{i,j,s}^1$ 。

在边缘 i 上新部署服务 s 的成本计算为: $C_{i,s}^1 = \min_{j \in \{j | x_{j,s}(t-1)=1\}} C_{i,j,s}^1$ 。

边缘 i 在 t 时刻关于服务部署方面的运营成本表达为:

$$c_i^1(t) = \sum_{s \in \mathcal{S}} C_{i,s}^1 \mathbb{I}_{\{1\}}(x_{i,s}(t) - x_{i,s}(t-1)), \quad (2)$$

系统模型和问题构建

任务卸载

如果任务从当前服务器 k 卸载到另一个服务器 m ，则会产生操作成本 $C_{k,m,s}^2$ ，其包括数据传输成本和由服务器间切换引起的服务中断成本。

边缘 i 在 t 时刻关于任务卸载方面的运营成本表示为：

$$c_i^2(t) = \sum_{s \in \mathcal{S}} C_{k,m,s}^2 \mathbb{I}_{\{1\}}(y_{i,k,s}(t-1)y_{i,m,s}(t)). \quad (3)$$

资源分配

改变资源分配决策可能会导致资源重新配置，并导致服务器启动和初始化造成的资源交付周期。由于关闭服务器比启动服务器快得多，我们将分配资源的增加作为资源重新配置的运营成本。

设 C_i^3 为计算资源的单位重新配置成本。边缘 i 在 t 时刻关于资源分配方面的成本表示为：

$$c_i^3(t) = \sum_{s \in \mathcal{S}} C_i^3 [z_{i,s}(t) - z_{i,s}(t-1)]^+. \quad (4)$$

系统模型和问题构建

问题构建

我们的目标是通过共同优化三层调度决策的时间尺度来最大化长期利润。

控制变量为 $a_i^1(t)$, $a_i^2(t)$, $a_i^3(t)$ 分别指示边缘服务器 i 上的当前部署、卸载和分配决策是否需要更新。

现有的调度算法充当内置规则, 为布局提供候选部署决策 $\tilde{x}_{i,s}(t)$, 卸载决策 $\tilde{y}_{i,j,s}(t)$ 和分配决策 $\tilde{z}_{i,s}(t)$

如果 $a_i^1(t) = 1$, 则服务器 i 将执行由现有规则生成的部署决策 $\tilde{x}_{i,s}(t)$ 。否则, 服务器 i 将会沿用之前的决策。因此, t 时刻边缘 i 的部署决策表达为:

$$x_{i,s}(t) = \Gamma^1(a_i^1(t)) = \begin{cases} \tilde{x}_{i,s}(t), & a_i^1(t) = 1, \\ \Gamma^1(a_i^1(t-1)), & a_i^1(t) = 0. \end{cases} \quad (5)$$

联合优化问题表达为:

$$\begin{aligned} & \tilde{\mathcal{P}}: \max \mathbb{E} \left[\sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} (g_i(t) - a_i^1(t)c_i^1(t) - a_i^2(t)c_i^2(t) - a_i^3(t)c_i^3(t)) \right] \\ & \text{s.t.} \quad a_i^1(t), a_i^2(t), a_i^3(t) \in \{0, 1\}, \forall t, i. \end{aligned} \quad (6)$$

问题分解的三层分层DRL

实现 (6) 中的目标依赖于执行三层紧密耦合的决策。这使得同时做出三个有效决策变得困难。为了方便学习，将原始问题分解为三个独立的子问题，并设计了一个三层分层深度强化学习 (HDRL) 框架，以协作的方式解决不同的子问题。

我们通过将总目标分解为不同调度层的子收益和子成本，将原始问题 $\tilde{\mathcal{P}}$ 解耦为子问题 $\mathcal{P}1, \mathcal{P}2, \mathcal{P}3$ 。

边缘-云服务部署子问题P1

虽然收入不能直接分解，但服务覆盖范围可以潜在地衡量当前部署决策对总收入的贡献，服务放置子问题可以表述为：

$$\begin{aligned} \mathcal{P}1 : \max \mathbb{E} & \left[\sum_{t=0}^{\infty} \alpha d(t) - \sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} a_i^1(t) c_i^1(t) \right] \\ \text{s.t.} \quad & a_i^1(t) \in \{0, 1\}, \forall t, i, \end{aligned} \quad (7)$$

其中， $d(t)$ 是 t 时刻部署的服务所覆盖的任务的计算需求量，系数 α 用于衡量服务范围的影响。

边缘-边缘任务卸载子问题P2

任务卸载的贡献度量为卸载的满足传输延迟预算的任务总数 $u(t)$ ，以及边缘服务器之间的负载方差 $v(t)$ 。

$$\begin{aligned} \mathcal{P}2 : \max \mathbb{E} & \left[\sum_{t=0}^{\infty} \beta u(t) - \sum_{t=0}^{\infty} \sigma v(t) - \sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} a_i^2(t) c_i^2(t) \right] \\ \text{s.t.} \quad & a_i^2(t) \in \{0, 1\}, \forall t, i, \end{aligned} \quad (8)$$

边缘内资源分配子问题P3

类似于任务卸载，资源分配的贡献度量为满足计算延迟预算的任务总数 $l(t)$

$$\begin{aligned} \mathcal{P}3 : \max \mathbb{E} & \left[\sum_{t=0}^{\infty} \phi l(t) - \sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} a_i^3(t) c_i^3(t) \right] \\ \text{s.t.} \quad & a_i^3(t) \in \{0, 1\}, \forall t, i. \end{aligned} \quad (9)$$

EdgeTimer设计

用于分布式调度的安全多客户端DRL

每个客户端 i 采用策略 $\pi_i(a_i(t)|o_i(t))$ 来根据部分观测 $o_i(t)$ 选择动作 $a_i(t)$ 。

Dec-POMDP的目标是找到所有客户端的联合策略 $\Pi = \{\pi_i(a_i(t)|o_i(t))|i \in \mathcal{N}\}$ 来最大化累计期望奖励

$$\max_{\Pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(\mathcal{S}(t), \Pi(\mathcal{A}(t)|\mathcal{O}(t)))], \quad (10)$$

其中, $\mathcal{S}(t)$, $\mathcal{A}(t)$, and $\mathcal{O}(t)$ 分别是时刻 t 处所有客户端的全局状态, 动作和观测。

在我们的问题中, DRL可能会产生与已执行操作冲突的**不安全操作**。例如将任务卸载到未部署服务的服务器上。

对此, 设计一个鉴别器辅助动作屏蔽方案, 以避免执行不安全的动作。对于不安全行为, 它们的相关逻辑, 即神经网络的原始输出, 被大负数替换, 因此选择这些不安全行为的概率几乎为零。这样, 代理将在应用之前用“更新决策”操作替换原始操作。

EdgeTimer设计

采用集中式训练和分散式执行方案，允许代理使用来自其他代理的非本地信息来简化**离线训练**，而**在线推理**过程中不允许使用额外的信息。

演员的分布式执行

对于每个客户端 $i \in N$, 他对应的演员 i 根据参数为 θ_i 的策略 π_{θ_i} 将本地观测 $o_i(t)$ 映射到动作 $a_i(t)$ 上。
为了最大化回报，演员 i 将根据策略梯度定理更新策略参数

$$\nabla_{\theta_i} J(\pi_{\theta_i}) = \mathbb{E}_{\pi_{\theta_i}} [\nabla_{\theta_i} \log \pi_{\theta_i}(a_i(t) | o_i(t)) A_i(\mathcal{O}(t), \mathcal{A}(t))], \quad (11)$$

其中, $A_i(\mathcal{O}(t), \mathcal{A}(t))$ 是接收全局观测的集中式优势函数,

输入为所有客户端的观测和动作 $\mathcal{O}(t) = (o_1(t), \dots, o_N(t))$, $\mathcal{A}(t) = (a_1(t), \dots, a_N(t))$, 输出为代理 i 的策略性能度量。
优势函数可以使用广义优势估计 (GAE) 来计算

$$A_i(\mathcal{O}(t), \mathcal{A}(t)) = \sum_{l=0}^{T-t} (\gamma \lambda)^l \bar{V}_i(t+l), \quad (12)$$

$$\bar{V}_i(t+l) = r_i(t+l) + \gamma V_i(\mathcal{O}(t+l+1)) - V_i(\mathcal{O}(t+l)), \quad (13)$$

$V_i(\mathcal{O}(t))$ 是一个值函数，它表示在策略 π_{θ_i} 下从全局观测 $\mathcal{O}(t)$ 开始的预期回报。

我们为客户端 i 设计了一个私有的评论家网络来近似 $V_i(\mathcal{O}(t))$ 。

评论家的集中式训练

具有参数 ω_i 的客户端 i 的评价 V_{ω_i} 可以通过最小化均方误差来训练，如下所示：

$$L(\omega_i) = (V_{\omega_i}(\mathcal{O}(t)) - \sum_{k=0}^{T-t} \gamma^k r_i(t+k))^2. \quad (14)$$

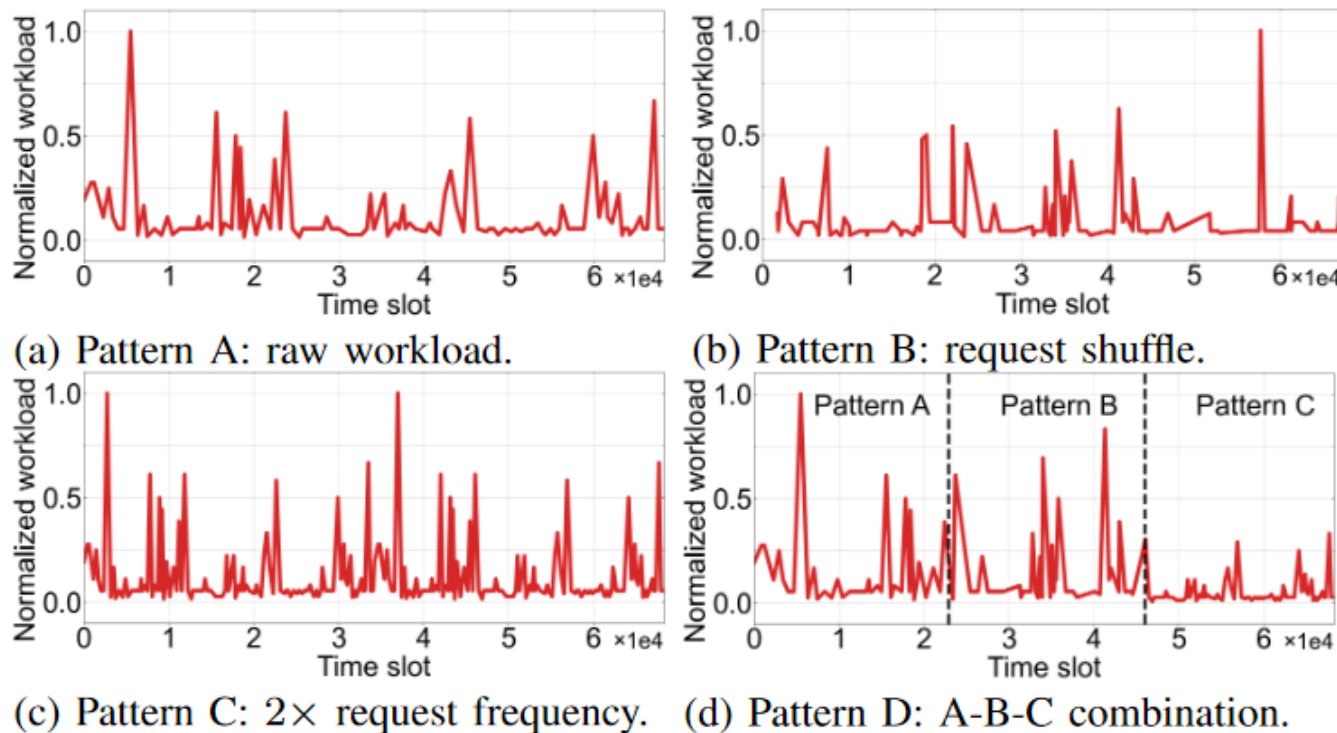
对比方案

静态单时间尺度 (SST), 在每个时隙更新三层调度决策;

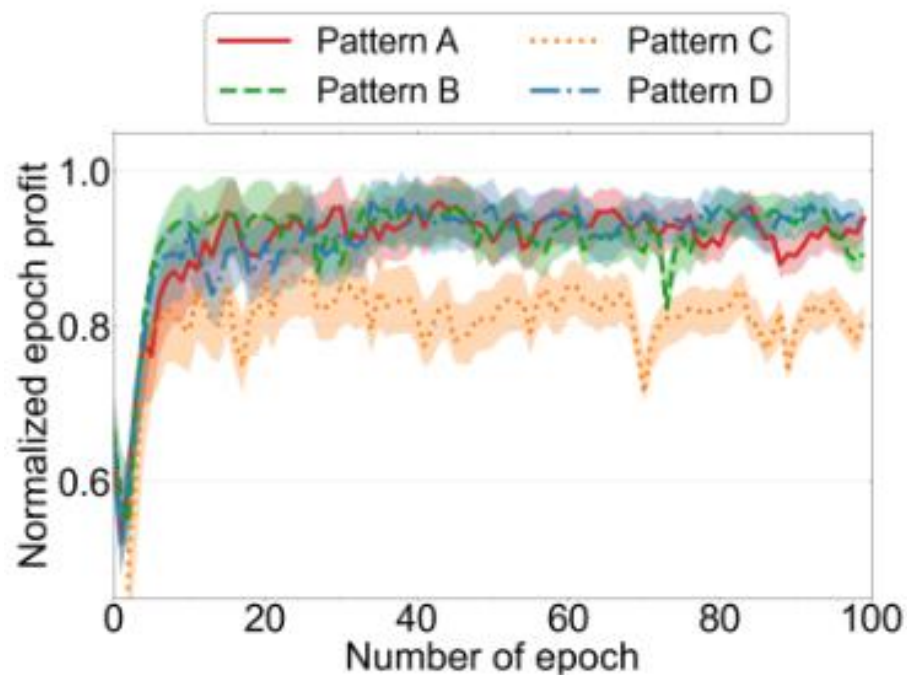
静态多时间尺度 (SMT), 通过固定的时间尺度比例更新每一层的决策;

延迟触发更新 (DT) 和工作负载触发更新 (WT) 分别在边缘中的延迟和等待工作负载超过相应阈值时更新边缘的决策。

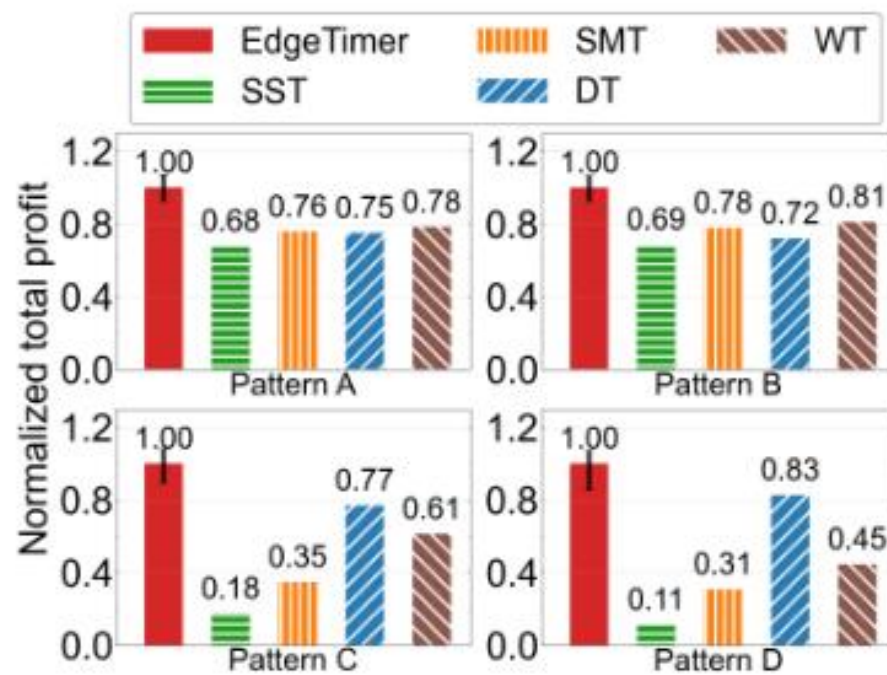
多模式负载



多模式负载下的利润

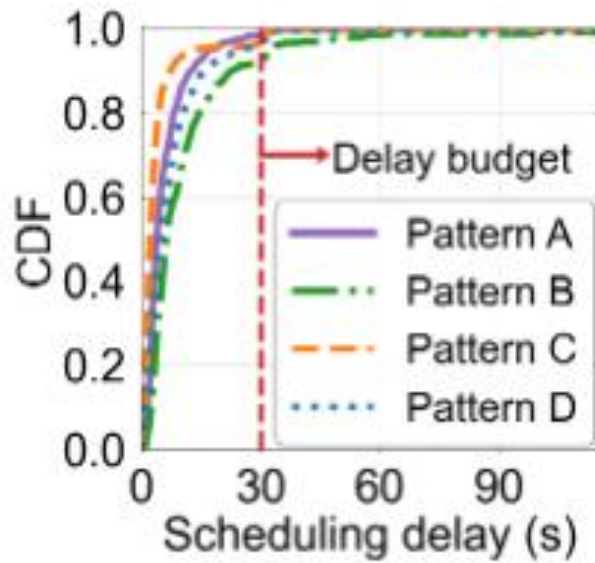


(a)

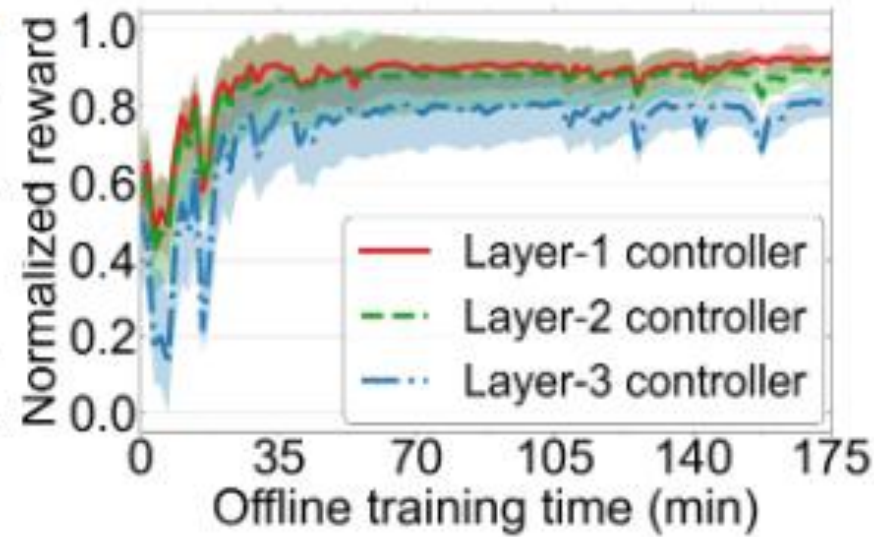


(b)

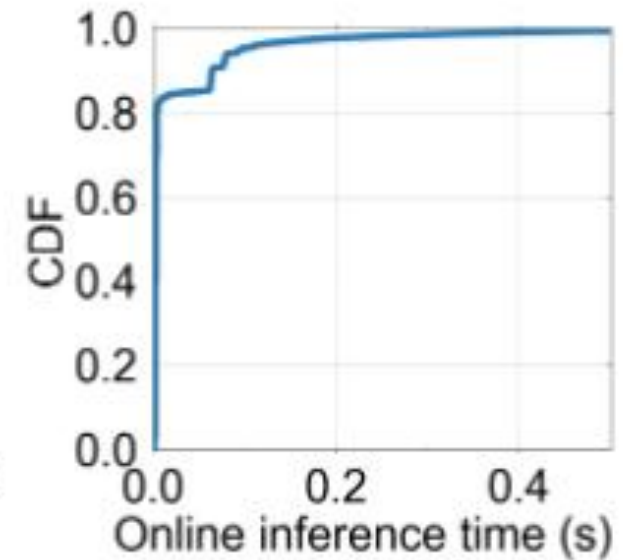
时间方面的表现



(a)



(b)



(c)

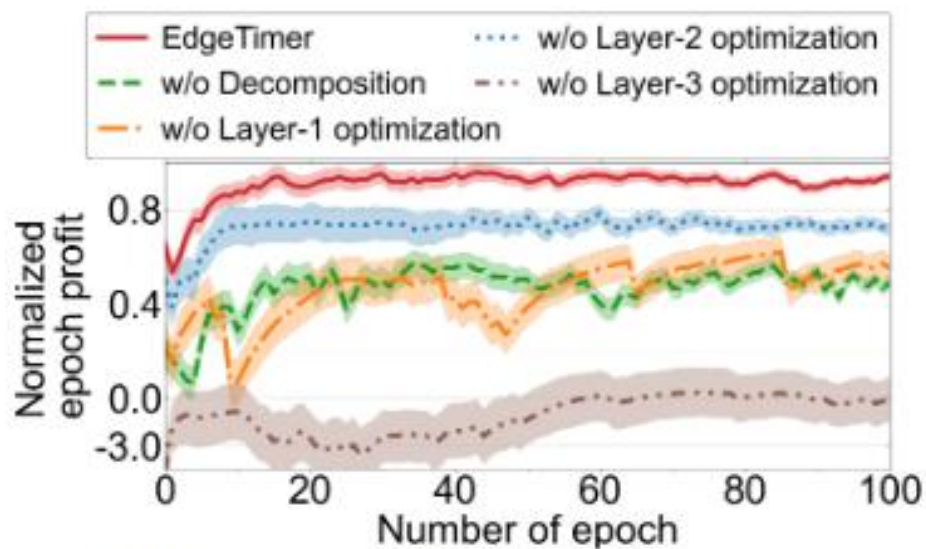
不同内置调度规则下的性能提升

TABLE II

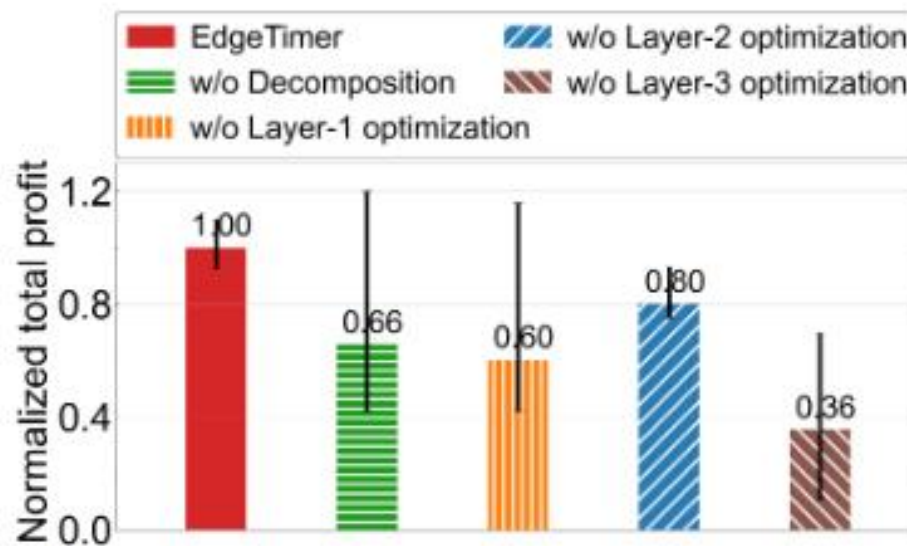
PROFIT GAINS COMPARED TO EXISTING APPROACHES UNDER DIFFERENT BUILT-IN SCHEDULING RULES IN PATTERN A

		Service placement rule - resource allocation rule								
		HPA - PF	HPA - RR	HPA - EA	Top-K - PF	Top-K - RR	Top-K - EA	AM - PF	AM - RR	AM - EA
Task offloading rule	MRP	1.73-2.43x	1.70-2.39x	1.66-2.81x	1.40-3.87x	1.39-3.86x	1.40-3.55x	1.30-1.55x	1.31-1.56x	1.28-1.47x
	LRP	1.71-2.39x	1.70-2.39x	1.72-2.94x	1.34-3.85x	1.36-3.90x	1.53-3.56x	1.26-1.53x	1.25-1.52x	1.26-1.48x
	RLP	1.72-2.41x	1.71-2.40x	1.72-2.92x	1.31-3.83x	1.32-3.87x	1.48-3.60x	1.26-2.37x	1.27-2.38x	1.33-1.49x
	SSP	1.38-1.99x	1.39-2.01x	1.41-2.98x	1.91-4.67x	1.92-4.70x	1.29-3.61x	1.28-3.89x	1.26-3.83x	1.23-2.07x
	RCRP	1.72-2.41x	1.73-2.43x	1.70-2.92x	1.32-3.82x	1.30-3.78x	1.57-3.63x	1.26-1.59x	1.26-1.60x	1.30-1.50x

分层DRL设计的影响



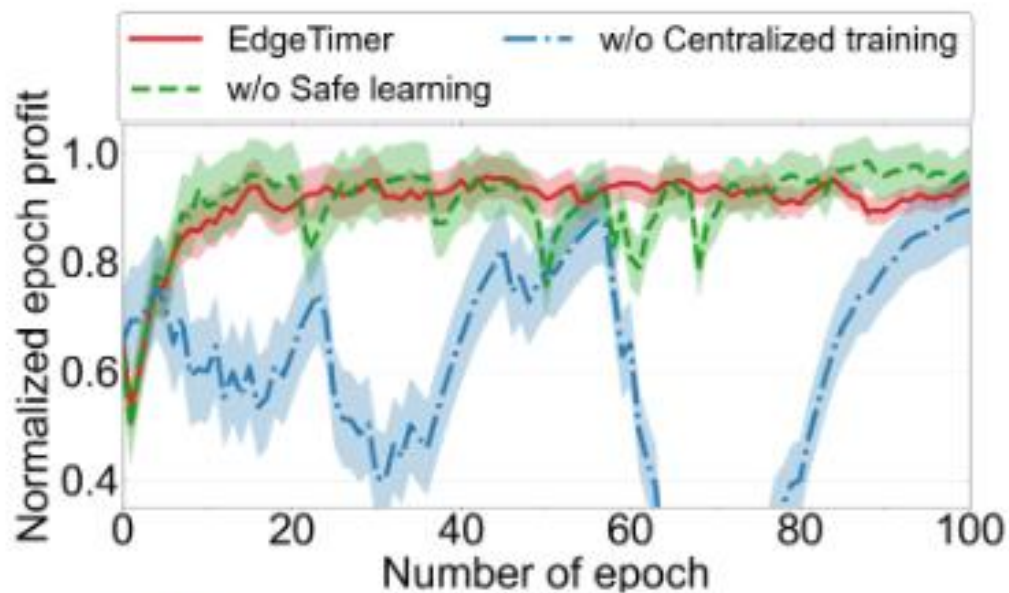
(a) Offline profits.



(b) Online profits.

其中，w/o表示without

安全多客户端DRL设计的影响



(a) Offline profits.



(b) Online profits.